

توثيق مشروع المنصة التعليمية الذكية

(1) مقدمة المشروع

هذا المشروع عبارة عن منصة تعليمية ذكية تعتمد أولاً على **RAG (Retrieval-Augmented Generation)** ثم تتوسع لاحقاً إلى **OCR** ومعالجة الصور ولقطات الشاشة. الهدف الأساسي هو تمكين الطالب من رفع ملفات تعليمية مثل PDF و Word، ثم السؤال عنها بالعربية أو الإنجليزية والحصول على إجابات دقيقة ومختصرة وموثوقة.

في المرحلة الأولى، يكون التركيز على بناء نظام RAG قوي جداً، لأن نجاحه يعني أن المنصة تستطيع فهم المحتوى النصي للمستندات والرد على الأسئلة والاختبارات والتلخيص بكفاءة عالية. بعد التأكد من أن RAG يعمل بشكل ممتاز مع ملفات PDF و Word، ينتقل المشروع إلى مرحلة OCR لدعم الصور ولقطات الشاشة والأسئلة المصورة.

(2) المشكلة التي يحلها المشروع

الطلاب والمستخدمون يواجهون مشاكل متكررة مثل:

- صعوبة البحث اليدوي داخل ملفات PDF الطويلة.
- ضياع الوقت في قراءة المستند كاملاً للحصول على معلومة صغيرة.
- الحاجة إلى تلخيص سريع وواضح.
- صعوبة حل أسئلة MCQ من داخل ملفات أو صور.
- الحاجة إلى دعم لغتين على الأقل: العربية والإنجليزية.
- ضعف أدوات الدردشة العادية في فهم المستندات التعليمية بدقة.

المشكلة الأساسية أن chatbot عادي لا يعرف المستندات الخاصة بالمستخدم، بينما RAG يسمح للنظام أن يبحث داخل المحتوى المرفوع ثم يجيب بناءً على ذلك المحتوى.

(3) الحل المقترح

الحل هو بناء منصة تعمل على مرحلتين:

المرحلة الأولى: RAG

- رفع ملفات PDF و Word.
- استخراج النص من الملفات.
- تقسيم النص إلى chunks ذكية.
- إنشاء embeddings متعددة اللغات.
- تخزين البيانات في vector database.
- استخدام hybrid retrieval يجمع بين BM25 والبحث الدلالي.
- استخدام reranking لتحسين ترتيب النتائج.
- توليد إجابات دقيقة بالعربية أو الإنجليزية.

المرحلة الثانية: OCR

- دعم رفع الصور ولقطات الشاشة.
- قراءة السؤال والخيارات من الصورة.
- إرسال النص المستخرج إلى المرسل/النموذج للإجابة.
- دعم layouts المعقدة عندما تكون الأسئلة داخل صورة أو لقطة شاشة.

(4) نطاق المشروع

في البداية

التركيز فقط على:

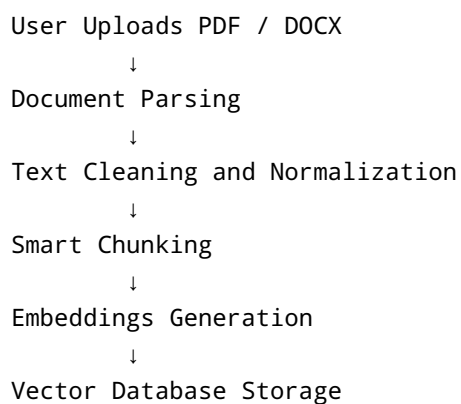
- PDF
- Word
- Arabic + English
- QA
- Summarization
- MCQ answering from text documents

لاحقًا

إضافة:

- OCR
- Screenshots
- Images of MCQ questions
- Table understanding
- Layout-aware document parsing

(5) المعمارية العامة للنظام



↓
Hybrid Retrieval (BM25 + Semantic Search)
↓
Reranking
↓
LLM Answer Generation
↓
Final Response in Arabic or English

وعند إضافة OCR لاحقًا:

User Uploads Image / Screenshot
↓
OCR / Layout Analysis
↓
Text Extraction
↓
Question + Choices Parsing
↓
RAG or LLM Answering
↓
Final Response

(6) لماذا نبدأ بـ RAG أولاً؟

لأن RAG هو الأساس الحقيقي للمشروع. إذا كان الاسترجاع ضعيفًا، فلن يكون الناتج قويًا مهما كان النموذج الذكي ممتازًا. لذلك الأولوية تكون لـ:

- جودة parsing.
- جودة chunking.
- جودة retrieval.
- جودة reranking.
- جودة بناء السياق.

إذا نجح RAG، فالمشروع سيعطي نتائج ممتازة في:

- الأسئلة النصية.
- الشرح.
- التلخيص.
- الاسترجاع من PDF و Word.
- دعم اللغتين.

7) المتطلبات الأساسية للمشروع

أ) المتطلبات الوظيفية

- رفع ملف PDF.
- رفع ملف Word.
- استخراج النص من الملف.
- دعم اللغة العربية والإنجليزية.
- سؤال وجواب على محتوى الملف.
- تلخيص المحتوى.
- دعم أسئلة MCQ من النصوص.
- عرض الإجابة مع مرجع أو سياق.

ب) المتطلبات غير الوظيفية

- سرعة مقبولة في الاستجابة.
- دقة عالية في الاسترجاع.
- دعم ملفات طويلة.
- قابلية التوسع.
- واجهة سهلة الاستخدام.
- قابلية إضافة OCR لاحقًا بدون إعادة بناء المشروع من الصفر.

8) التقنيات المقترحة

Backend

- Python
- FastAPI

Frontend

- Next.js
- React
- TypeScript
- Tailwind CSS

Document Parsing

- PyMuPDF
- python-docx
- Docling أو مكتبة مشابهة لاحقًا

Embeddings

- e5 multilingual أو BGE-M3 مثل multilingual embeddings

Vector Database

• Weaviate أو Pinecone أو Qdrant

Retrieval

- Hybrid Search
- BM25
- Semantic Search

Reranking

• BGE Reranker أو نموذج reranking مناسب

LLM

• نموذج لغوي API أو نموذج محلي لاحقًا

OCR لاحقًا

- PaddleOCR
- Layout-aware OCR

(9) استراتيجية الـ RAG

9.1 Parsing

استخراج النص من PDF و Word مع الحفاظ على أكبر قدر ممكن من البنية العامة.

9.2 Cleaning

تنظيف النص من:

- الفراغات الزائدة.
- الرموز غير المفيدة.
- مشاكل الترميز.
- التكرارات.

9.3 Chunking

تقسيم النص إلى chunks ذكية، وليس تقسيمًا عشوائيًا.

الأفضل أن يكون التقسيم حسب:

- العناوين.
- الفقرات.
- الأقسام.

- الجداول.
- الأسئلة والاختيارات معًا.

Embeddings 9.4

تحويل النص إلى embeddings متعددة اللغات حتى يدعم العربية والإنجليزية.

Retrieval 9.5

استخدام:

- BM25 للمطابقة الحرفية.
- Semantic Search لفهم المعنى.
- Hybrid Search لدمج الطريقتين.

Reranking 9.6

ترتيب النتائج بعد الاسترجاع لتحديد أفضل المقاطع المرتبطة بالسؤال.

Answer Generation 9.7

إرسال أفضل المقاطع إلى النموذج اللغوي ليولد إجابة دقيقة ومبنية على السياق المسترجع.

10) استراتيجية دعم اللغتين

المشروع يجب أن يدعم:

- العربية.
- الإنجليزية.

ولذلك يجب الانتباه إلى:

- استخدام embeddings متعددة اللغات.
- تنظيف النص العربي بشكل جيد.
- دعم UTF-8.
- عدم كسر الكلمات العربية أثناء chunking.
- تجربة retrieval بلغتين منفصلتين ومشاركتين.

11) خطة OCR المستقبلية

في المرحلة الثانية، بعد نجاح RAG، نضيف OCR لدعم:

- لقطة شاشة.
- صورة سؤال.
- سؤال مع خيارات داخل صورة.

- استخراج النص من الصور.
- فهم layout بشكل أفضل.

الهدف من OCR

عندما يرفع المستخدم صورة فيها سؤال MCQ، يتم: 1. قراءة النص داخل الصورة. 2. استخراج السؤال. 3. استخراج الخيارات. 4. تمريرها للنموذج. 5. إخراج الإجابة الصحيحة مع شرح مختصر.

12) لماذا OCR يأتي بعد RAG؟

لأن OCR أصعب وأكثر حساسية من النص الخام. لو بدأت به من البداية ستدخل في تعقيدات كثيرة قبل أن تتأكد أن الـ core system نفسه يعمل. لذلك الترتيب الصحيح هو:

1. RAG.
2. اختبار PDF.
3. اختبار Word.
4. تحسين retrieval.
5. ثم OCR.

13) مخرجات النظام المتوقعة

في مرحلة RAG

- إجابة مباشرة.
- شرح مختصر.
- تلخيص.
- إجابة مع سياق.
- دعم أسئلة MCQ النصية.

في مرحلة OCR

- نص مستخرج من الصورة.
- سؤال مع خيارات.
- إجابة على سؤال مصور.
- دعم screenshots التعليمية.

14) هيكل الملفات المقترح للمشروع

يمكنك تنظيم المشروع بهذه الملفات:

- README.md — تعريف عام بالمشروع.
- ARCHITECTURE.md — المعمارية العامة.
- ROADMAP.md — خطة المراحل.

- RAG_STRATEGY.md — تفاصيل الاسترجاع.
- CHUNKING_GUIDE.md — طريقة التقسيم.
- OCR_PLAN.md — خطة OCR المستقبلية.
- DATAFLOW.md — تدفق البيانات داخل النظام.
- API_SPEC.md — وصف الـ APIs.
- EVALUATION.md — طريقة قياس الجودة.

هل تكتب skill.md ؟

إذا كنت تقصد ملفًا داخليًا يشرح مهارات المشروع أو قواعده، فليس ضروريًا أن يكون اسمه skill.md . الأفضل أن تستخدم أسماء أوضح مثل:

- RAG_STRATEGY.md
- OCR_PLAN.md
- PROJECT_GUIDE.md

أما إذا كان قصدك ملفًا يشبه دليل داخلي للفريق أو للمطورين، فيمكنك كتابته، لكن ليس شرطًا أن يكون بهذا الاسم. الأهم أن يكون واضحًا ومرتبًا داخل مجلد docs/ .

(15) خطة التنفيذ العملية

المرحلة 1: MVP RAG

- صفحة رفع ملف.
- استخراج النص.
- تخزين الـ chunks.
- سؤال وجواب.
- دعم العربية والإنجليزية.

المرحلة 2: تحسين الجودة

- Hybrid search.
- Reranking.
- Better chunking.
- Context window management.

المرحلة 3: الاختبارات

- PDFs نصية.
- PDFs طويلة.
- Word files.
- أسئلة مباشرة.
- MCQ من النص.

المرحلة 4: OCR

- صور.
- Screenshots.

- MCQ images.
- Layout detection.

(16) تعريف المشكلة والحل بشكل مختصر للوثيقة الرسمية

المشكلة

المستخدم لديه ملفات تعليمية كثيرة، ويحتاج نظامًا يفهم هذه الملفات ويجيب عن محتواها بسرعة ودقة، مع دعم العربية والإنجليزية.

الحل

بناء منصة تعليمية ذكية تعتمد على RAG أولاً، ثم OCR لاحقًا، بحيث تسترجع المعلومات من الملفات المرفوعة وتجيب عنها بطريقة دقيقة ومفيدة.

(17) معايير النجاح

يعتبر المشروع ناجحًا عندما يستطیع:

- الإجابة عن الأسئلة من الملفات بدقة.
- تلخيص المحتوى بشكل جيد.
- العمل بالعربية والإنجليزية.
- التعامل مع PDF و Word بشكل ثابت.
- دعم OCR لاحقًا بدون كسر النظام الأساسي.

(18) توصية نهائية

ابدأ بالـ RAG فقط. لا تخط مع OCR من البداية. ابن أساسًا قويًا، واختبره جيدًا، ثم أضف OCR لاحقًا. هذا هو المسار الاحترافي والأكثر أمانًا للمشروع.